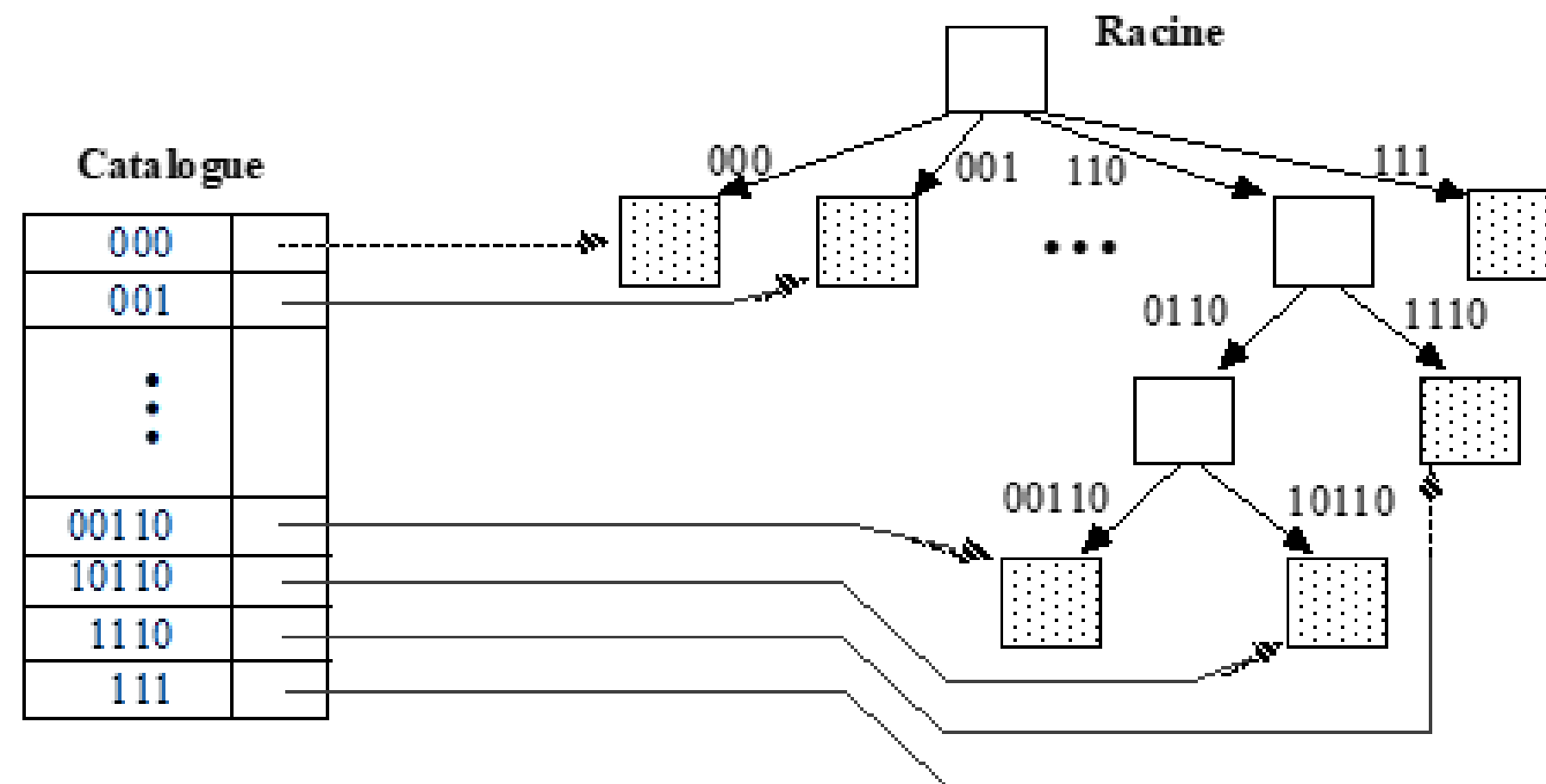


HASHING DINÁMICO

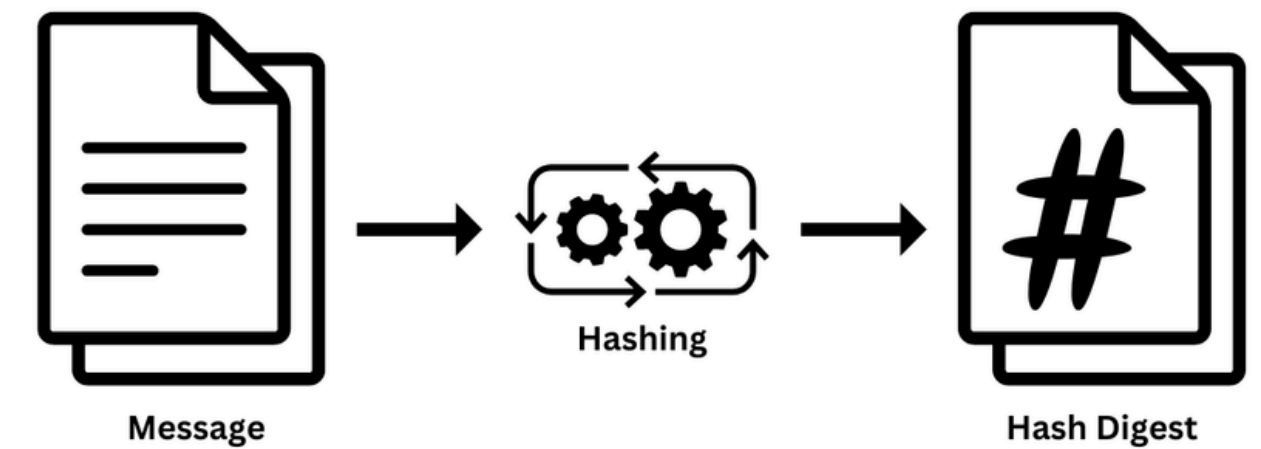
Laura Vanessa Botero Gil
Duban Lopez Sanchez
Carolina Arboleda Guzmán
Carol Juliana Henao Causil

Estructura de datos y lab.



INTRODUCCIÓN

El hashing constituye una de las técnicas fundamentales para la organización y acceso eficiente a datos en sistemas informáticos. Su principio básico consiste en transformar una clave de búsqueda en una dirección física mediante una función hash, permitiendo teóricamente el acceso directo a los registros.



HASHING

FUNDAMENTOS DEL HASHING Y PROBLEMÁTICA DEL ENFOQUE ESTÁTICO



Facultad de ingeniería

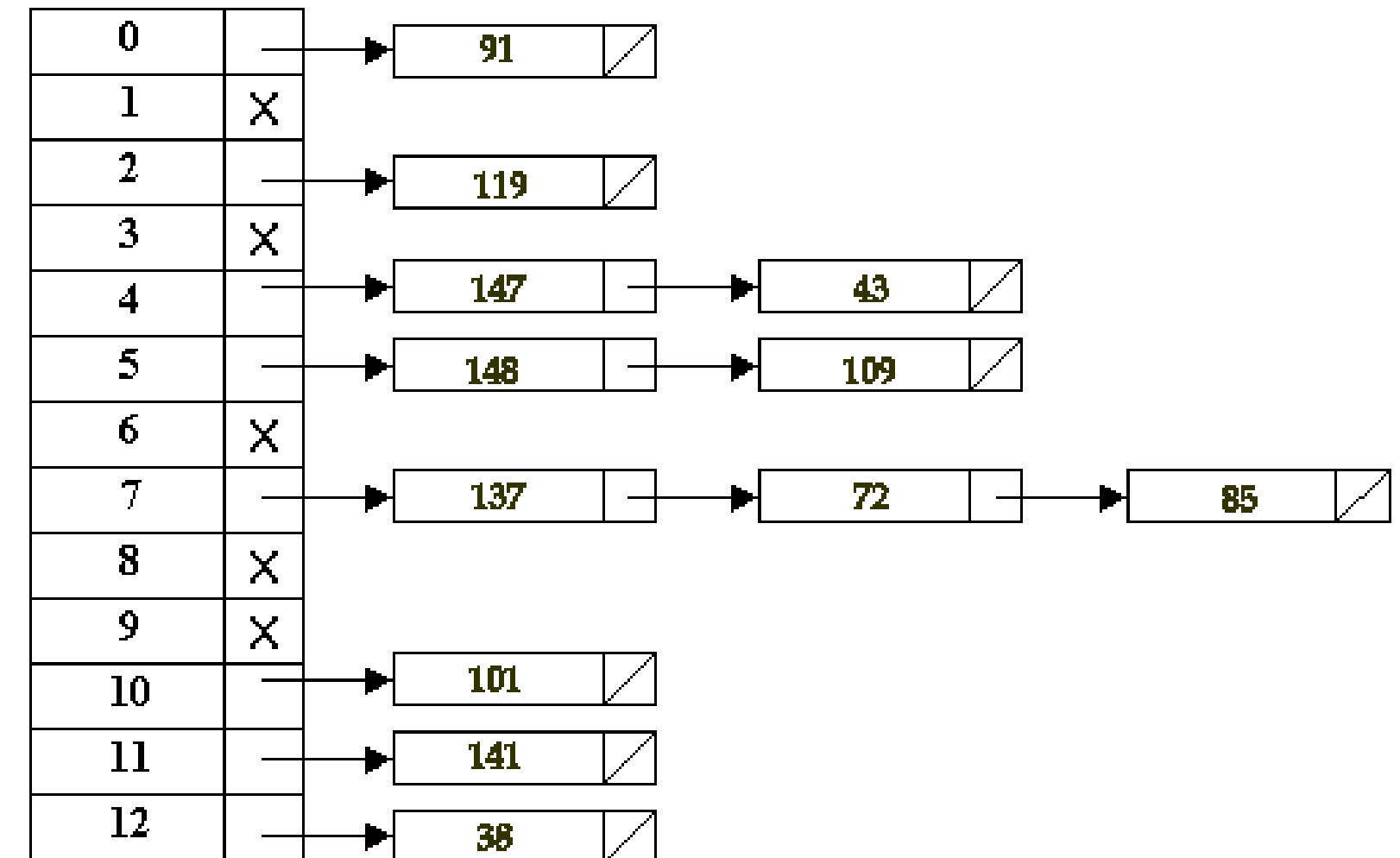
- Bucket (Cubeta): Unidad básica de almacenamiento que puede contener múltiples registros.
- Factor de Carga (LF): Relación entre registros almacenados y capacidad total.
Fórmula: $LF = N / (M \times Bkfr)$
 - N: Número de registros
 - M: Número de buckets
 - Bkfr: Factor de bucket
- Colisión: Situación cuando dos o más claves diferentes producen el mismo valor hash.

TRES PROBLEMAS FUNDAMENTALES DEL HASHING ESTÁTICO

1. Asignación Fija de Espacio: El número de buckets se determina al inicio y permanece constante.
2. Degradación del Rendimiento: Al insertar registros, el factor de carga aumenta, incrementando colisiones.
 - **Búsqueda exitosa:** $TF = s + r + dtt + (x/2) \times (s + r + dtt)$
 - **Búsqueda infructuosa:** $TF = s + r + dtt + x \times (s + r + dtt)$(Donde x es la longitud promedio de la cadena de desbordamiento)
3. Reorganización Costosa:
 - Asignar nuevo espacio.
 - Nueva función hash.
 - Reinsertar todos los registros existentes (Operación $O(n)$).

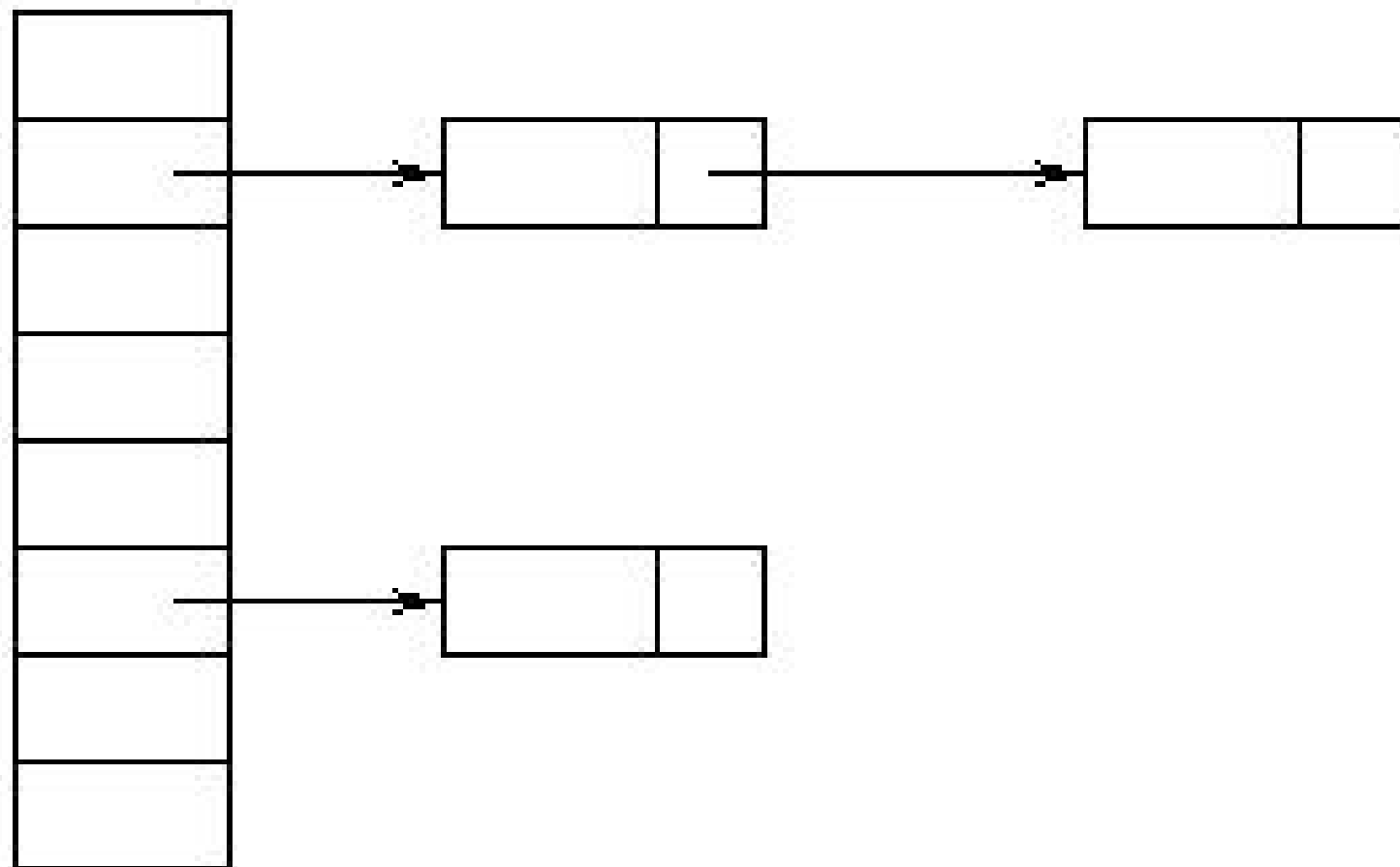
EL ENCADENAMIENTO EN EL CONTEXTO DEL HASHING DINÁMICO

El encadenamiento (chaining) es una de las técnicas más fundamentales para manejar colisiones en tablas hash, y adquiere características particulares cuando se combina con el hashing dinámico.



Resultado usando LIFO.

¿QUÉ ES EL ENCADENAMIENTO?



Cuando dos o más claves producen el mismo índice hash, necesitamos una forma de almacenar ambas. El encadenamiento resuelve esto haciendo que cada celda de la tabla apunte a una lista enlazada de todos los elementos que colisionaron en ese índice.

El factor de carga mide qué tan llena está la tabla:

$$\lambda = n / m$$

n = número de elementos

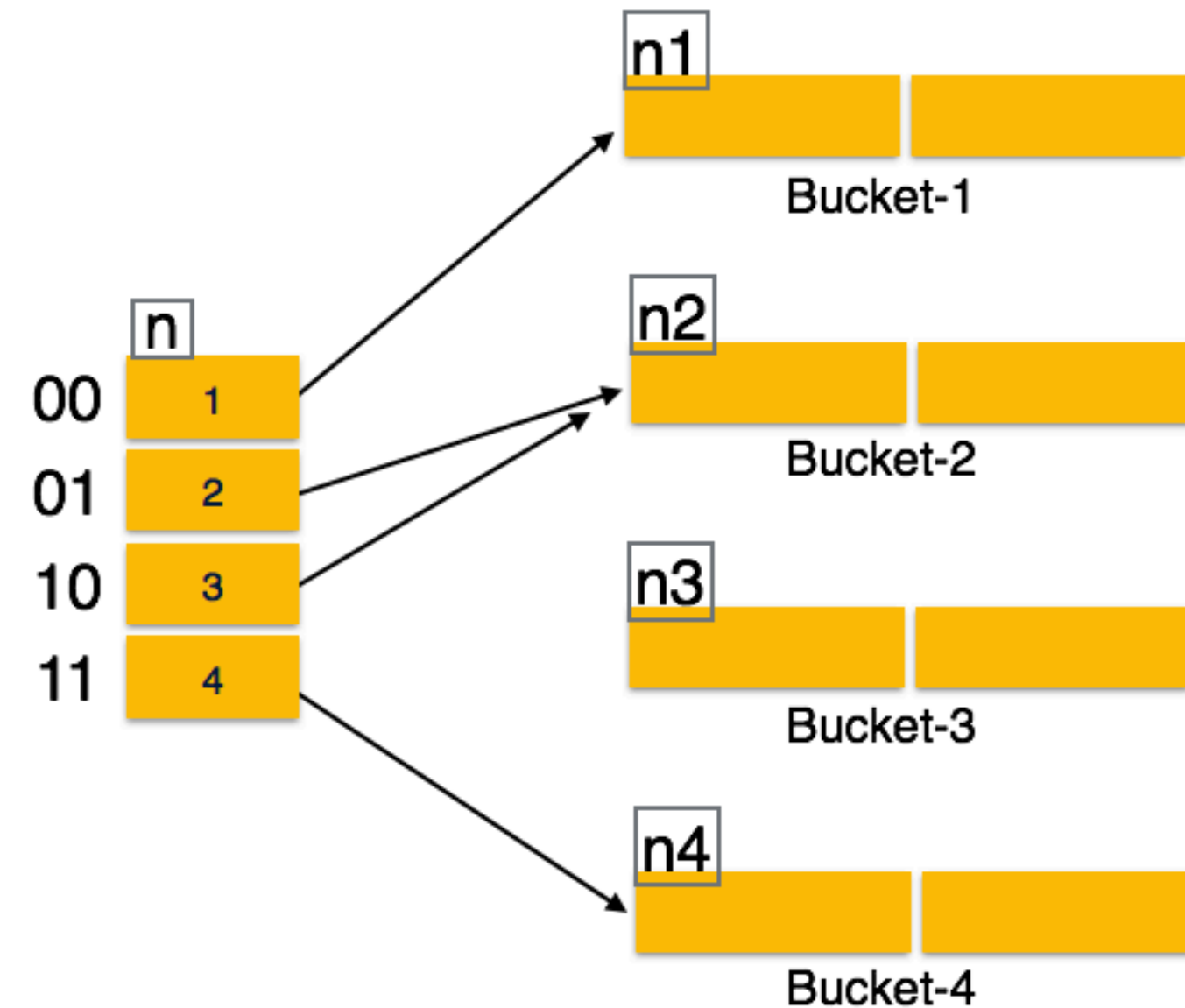
m = tamaño de la tabla

ENCADENAMIENTO + HASHING DINAMICO: CÓMO INTERACTÚAN

Rehashing (Redimensionamiento)

Cuando a supera un umbral, la tabla:

- Crea un nuevo arreglo de mayor tamaño.
- Recorre todas las listas enlazadas de cada celda
- Reinserta cada elemento usando la nueva función hash



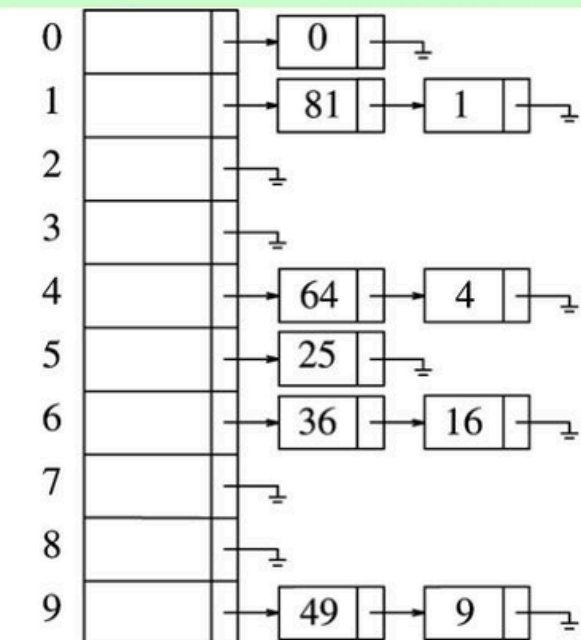
FLUJO DEL PROCESO

1. Ocorre una colisión
2. Se aplica encadenamiento
3. La lista crece
4. Se detecta saturación
5. El hashing dinámico reorganiza
6. Mejora el rendimiento

Chaining strategy: maintains a linked list at every hash index for collided elements

Insertion sequence: { 0 1 4 9 16 25 36 49 64 81 }

- Hash table T is a vector of linked lists
 - Insert element at the head (as shown here) or at the tail
- Key k is stored in list at $T[h(k)]$
- E.g., TableSize = 10
 - $h(k) = k \bmod 10$
 - Insert first 10 perfect squares



VARIANTES DE HASHING DINÁMICO CON ENCADENAMIENTO

Hashing Extensible

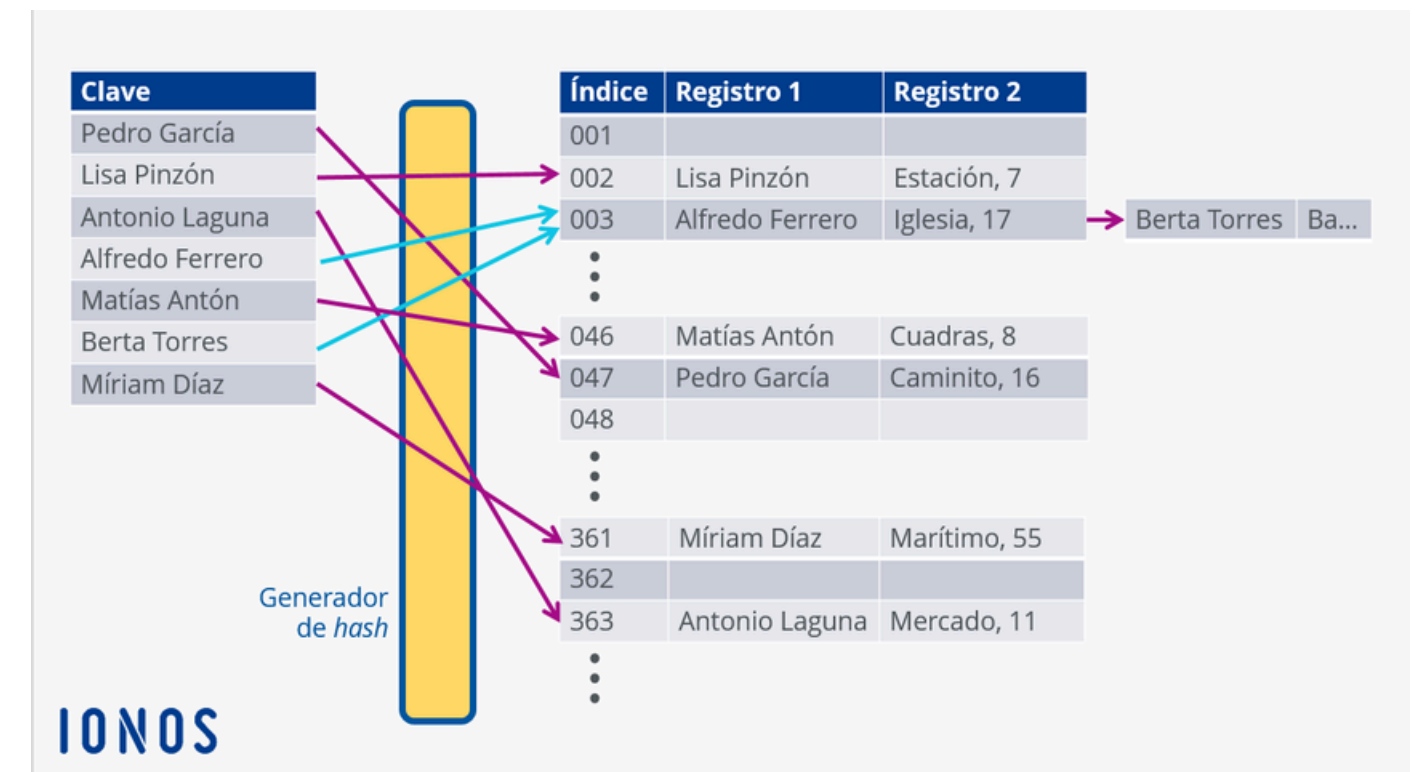
Usa un directorio de punteros a "buckets". Cuando un bucket se llena, solo ese bucket se divide, sin rehashear toda la tabla. El encadenamiento ocurre dentro de cada bucket.

Hashing Lineal

Divide los buckets de forma secuencial y controlada, usando un puntero de división. El encadenamiento maneja el desbordamiento local de cada bucket.

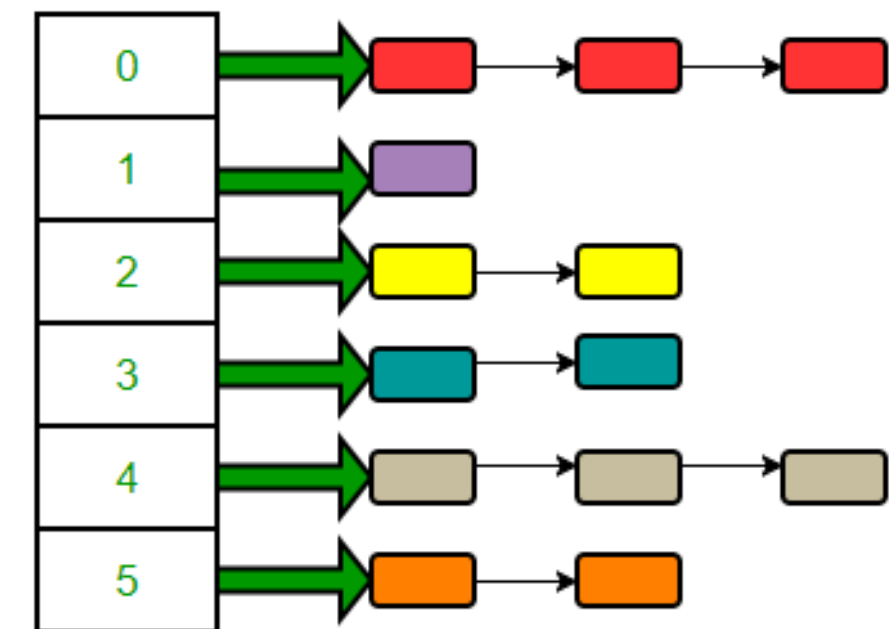
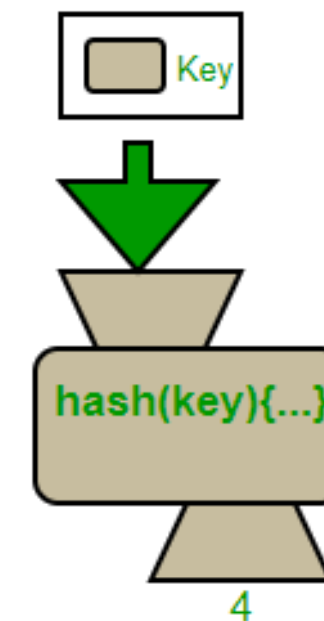
VENTAJAS DEL ENCADENAMIENTO EN CONTEXTO DINÁMICO

- **Tolerancia a a alto:** Funciona bien incluso con muchas colisiones antes del redimensionamiento
- **Eliminación simple:** No deja "huecos fantasma" como el direccionamiento abierto
- **Redimensionamiento gradual posible:**
En hashing lineal, se puede dividir de a un bucket a la vez



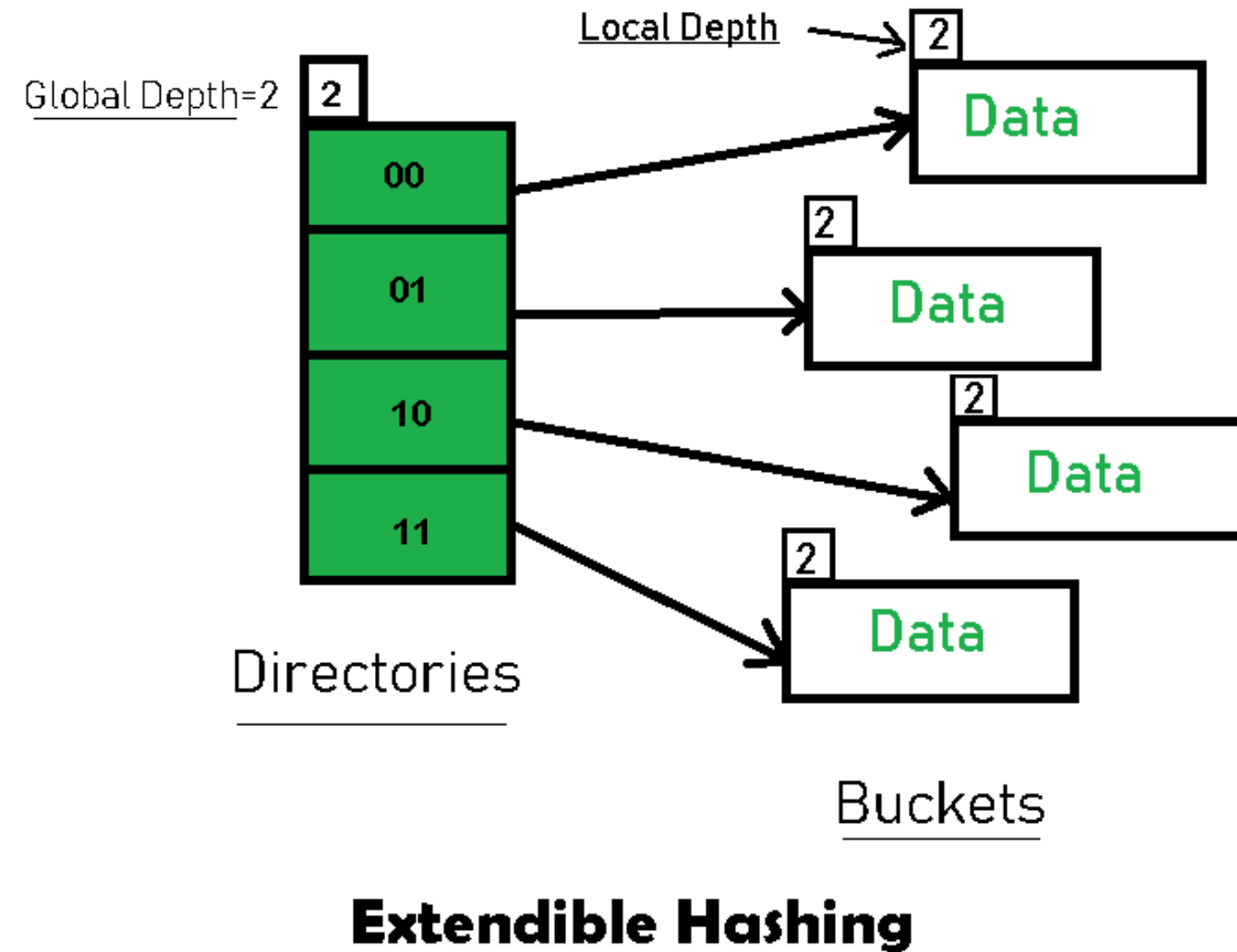
DESVENTAJAS

- Memoria extra por los punteros de las listas enlazadas
- Peor localidad de caché que el direccionamiento abierto
- El rehashing recorre todas las listas, lo que tiene costo $O(n + m)$



HASHING EXTENSIBLE

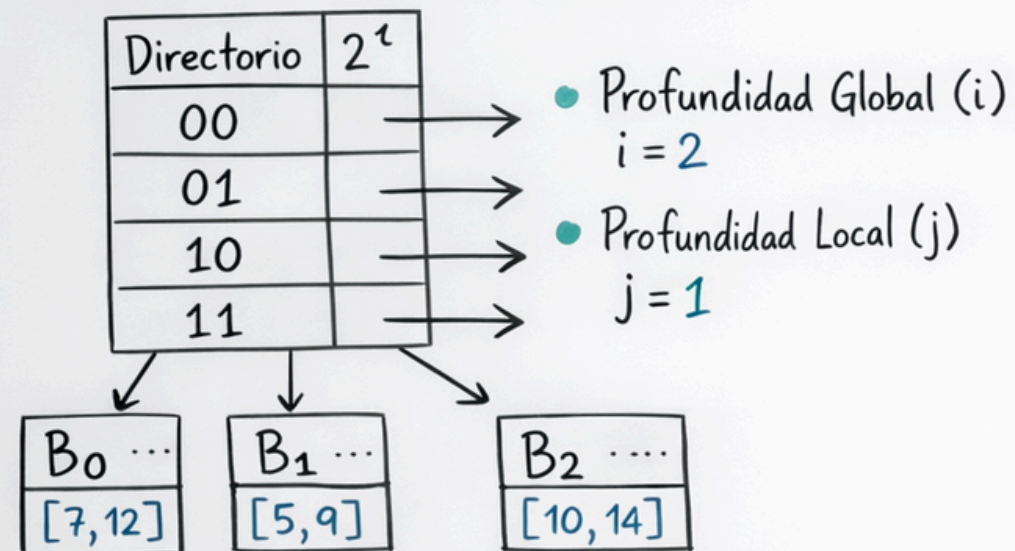
El hashing extensible, introducido por Fagin, Nievergelt, Pippenger y Strong en 1979, representa uno de los primeros y más influyentes esquemas de hashing dinámico. Su característica distintiva es el uso de un directorio en memoria principal que actúa como nivel de indirección hacia los buckets en disco.



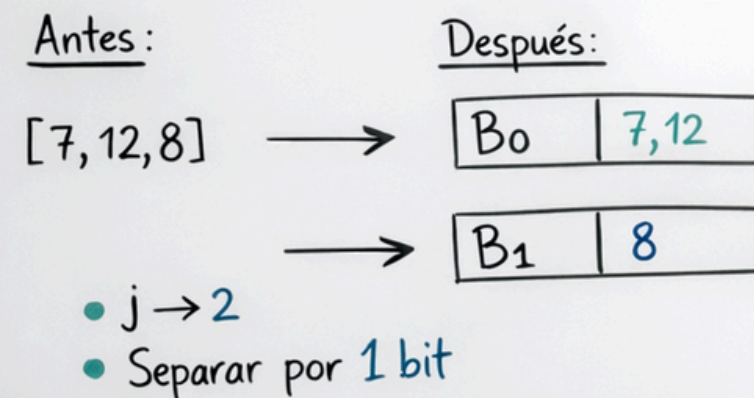
ESTRUCTURA DE DATOS

1. Directorio
2. Profundidad Global
3. Profundidad Local
4. Buckets de Datos

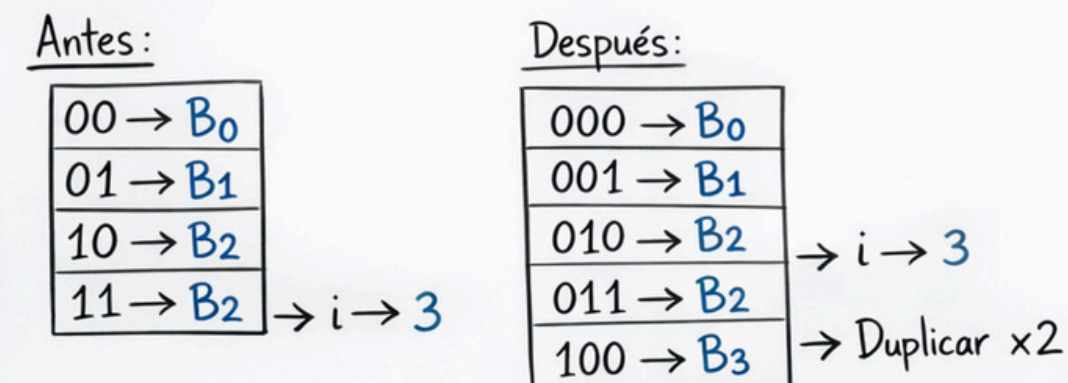
1. Estructura de Hashing Extensible



2. División de Buckets ($j < i$)



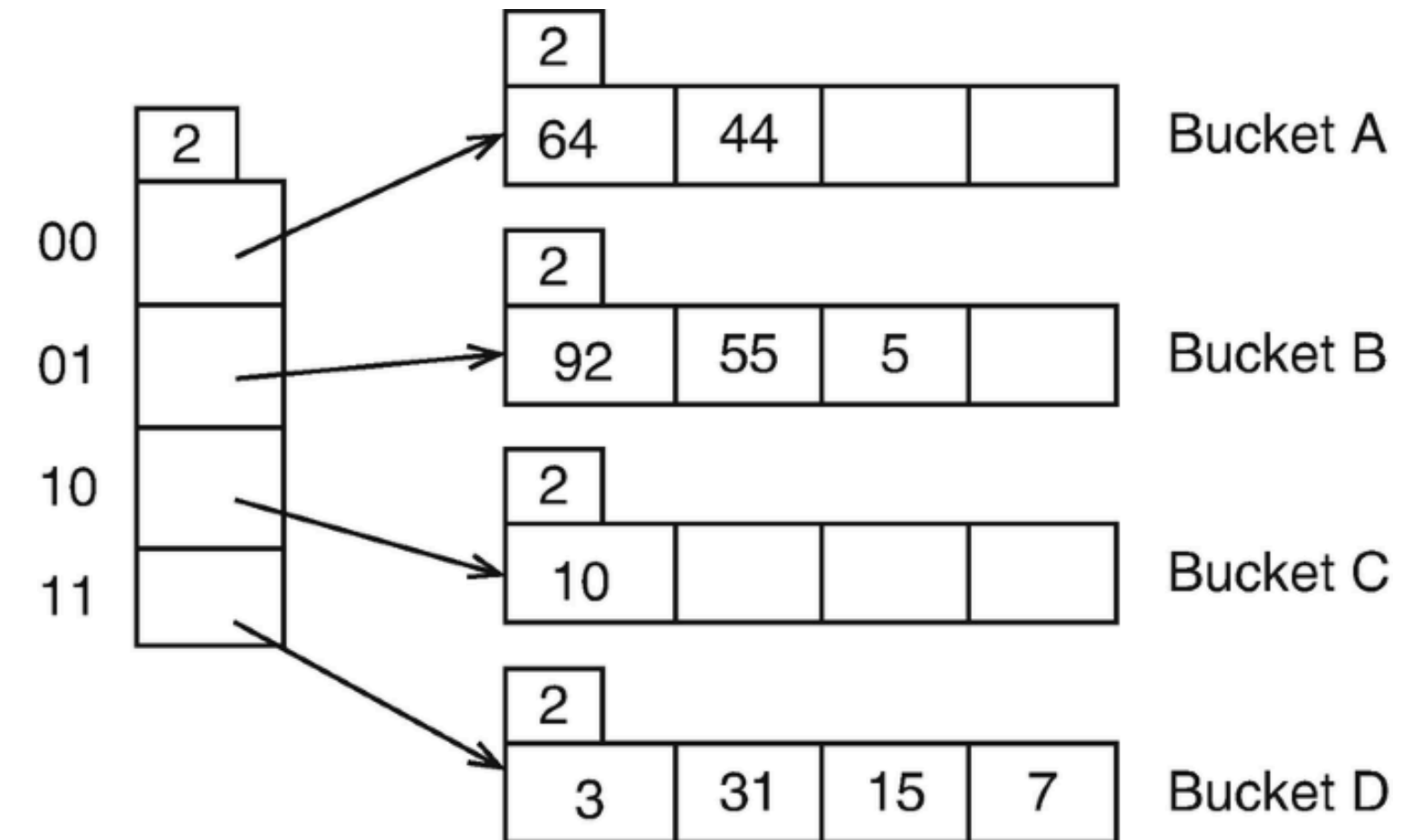
3. Duplicación del Directorio ($j = i$)



MECANISMOS DE CRECIMIENTO

El crecimiento en hashing extensible sigue un proceso incremental basado en bits:

1. Inserción sin desbordamiento:
2. Inserción con desbordamiento (Caso 1: $j < i$):
3. Inserción con desbordamiento (Caso 2: $j = i$):



EJEMPLO

Supongamos que cada bucket puede almacenar máximo 2 elementos.

1. Insertamos los valores 1 y 2.

Todo funciona normalmente.

2. Ahora insertamos el valor 3 y el bucket se llena.

Como la profundidad local es igual a la global, el directorio se duplica.

3. Luego, los datos se redistribuyen en nuevos buckets.

De esta manera, el sistema crece sin afectar el rendimiento.

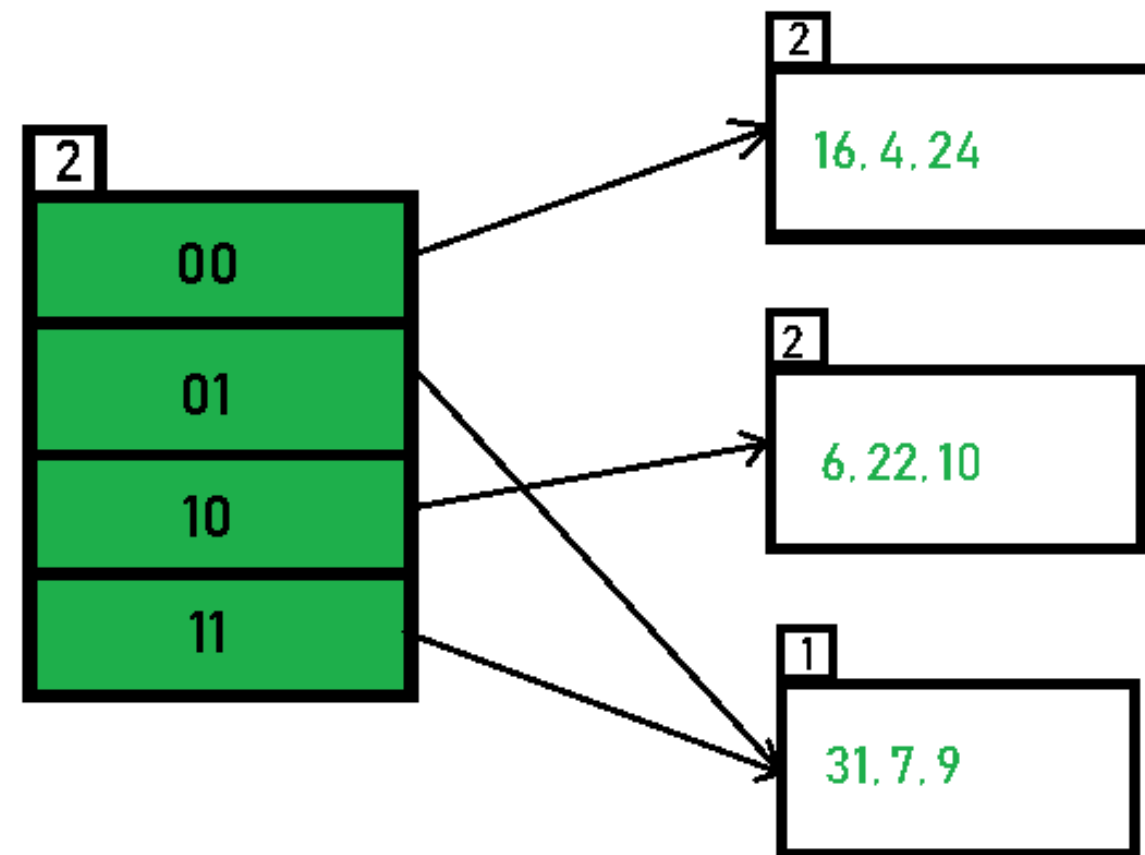
EJERCICIO DE HASHING EXTENSIBLE

Tenemos un sistema de hashing extensible con las siguientes condiciones:

- Capacidad del bucket: 2
- Profundidad global: $i = 1$

Insertar: 1, 2, 3

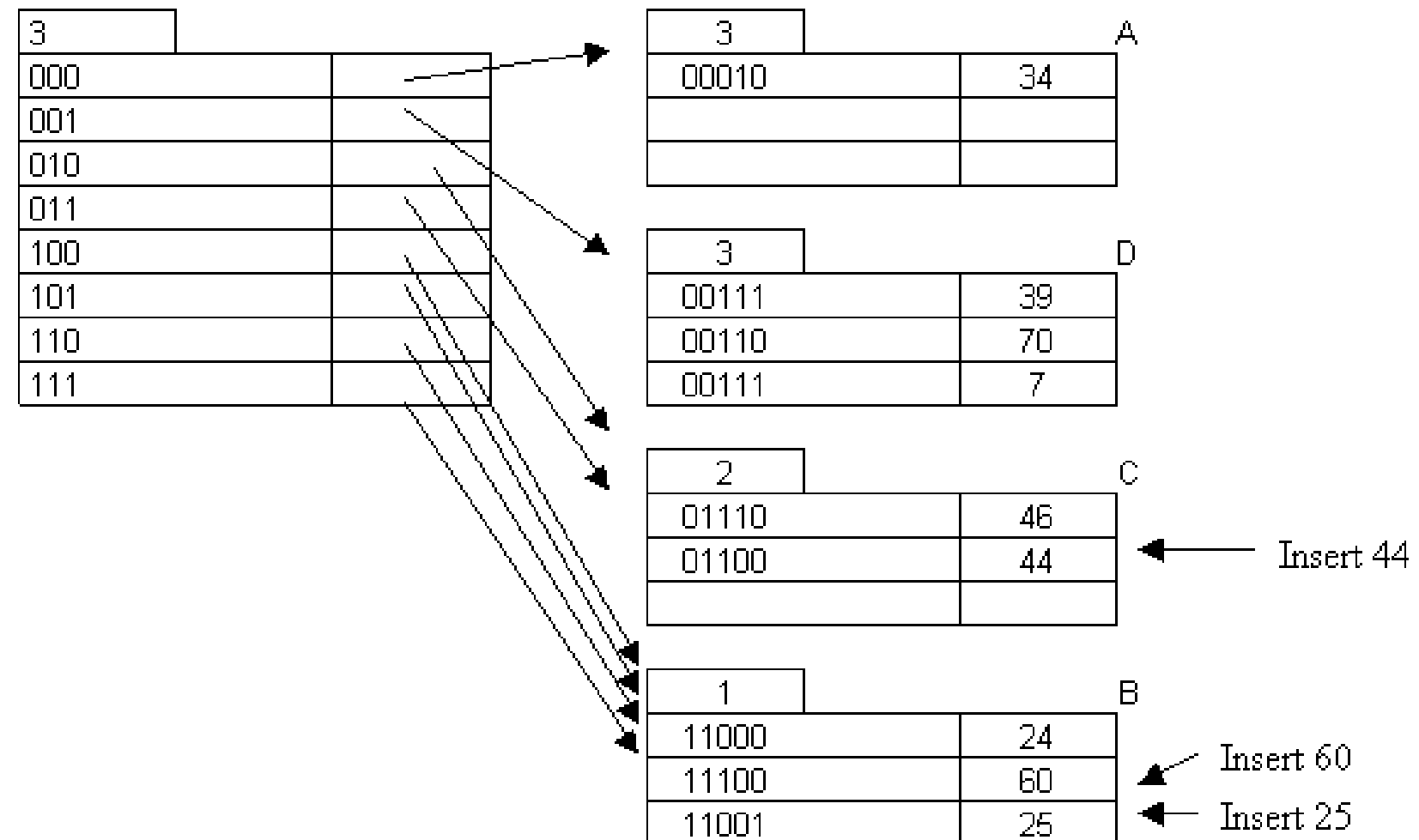
[Link del ejercicio resuelto](#)



$Hash(31) = 1111$
 $Hash(7) = 111$
 $Hash(9) = 1001$

VENTAJAS

1. Acceso en 1-2 operaciones de disco.
2. Reorganización localizada.
3. Factor de carga estable.
4. No requiere buckets de desbordamiento.



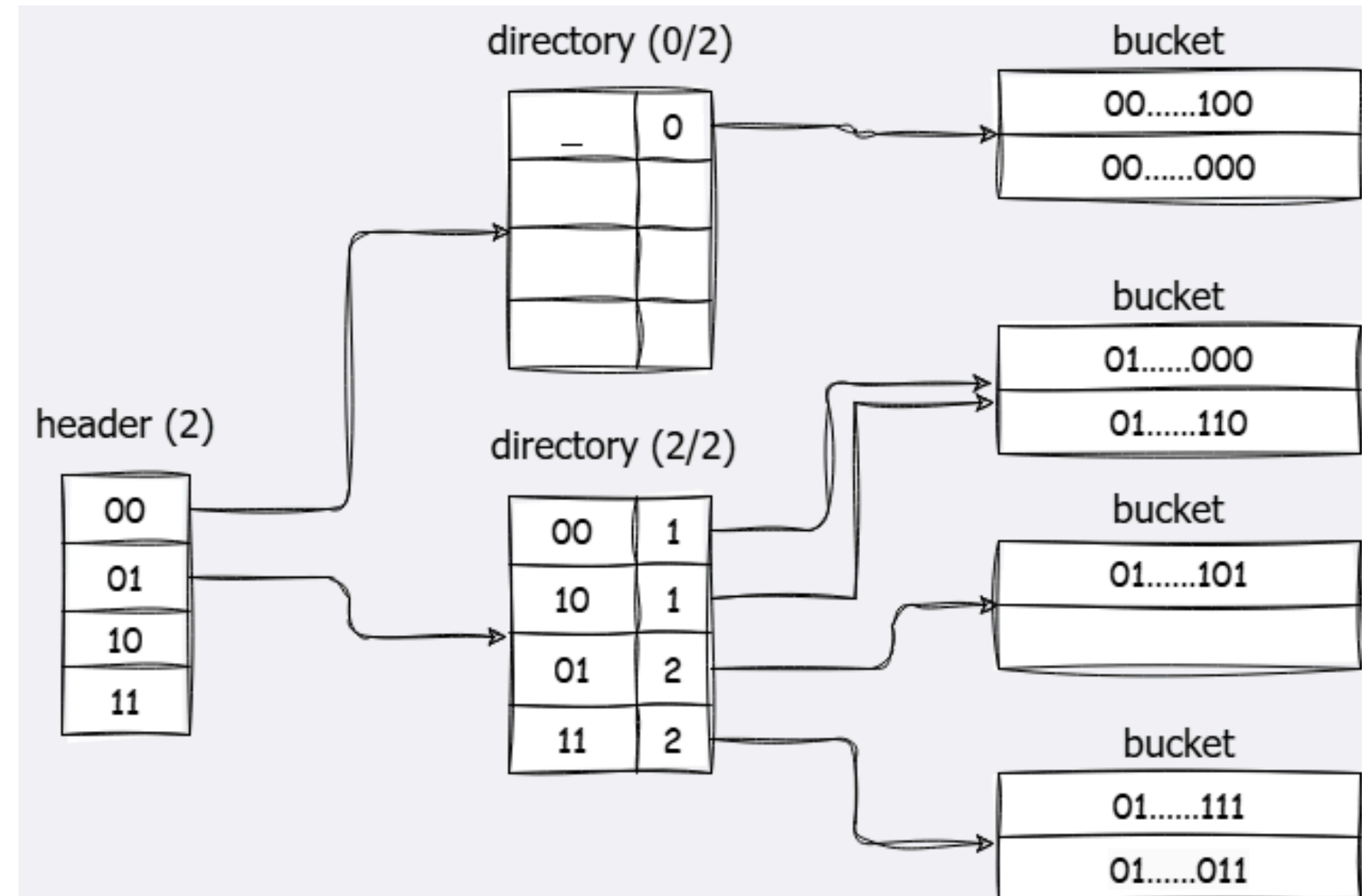
DESVENTAJAS

1. Crecimiento exponencial del directorio.
2. Dependencia del directorio en memoria.
3. Complejidad de implementación.

ANALISIS DEL RENDIMIENTO

Estudios comparativos indican que el hashing extensible ofrece:

- Utilización de almacenamiento ligeramente superior al hashing lineal (aproximadamente 5%)
- 1-2 accesos a disco garantizados para cualquier operación
- Rendimiento muy estable bajo diversas distribuciones de datos

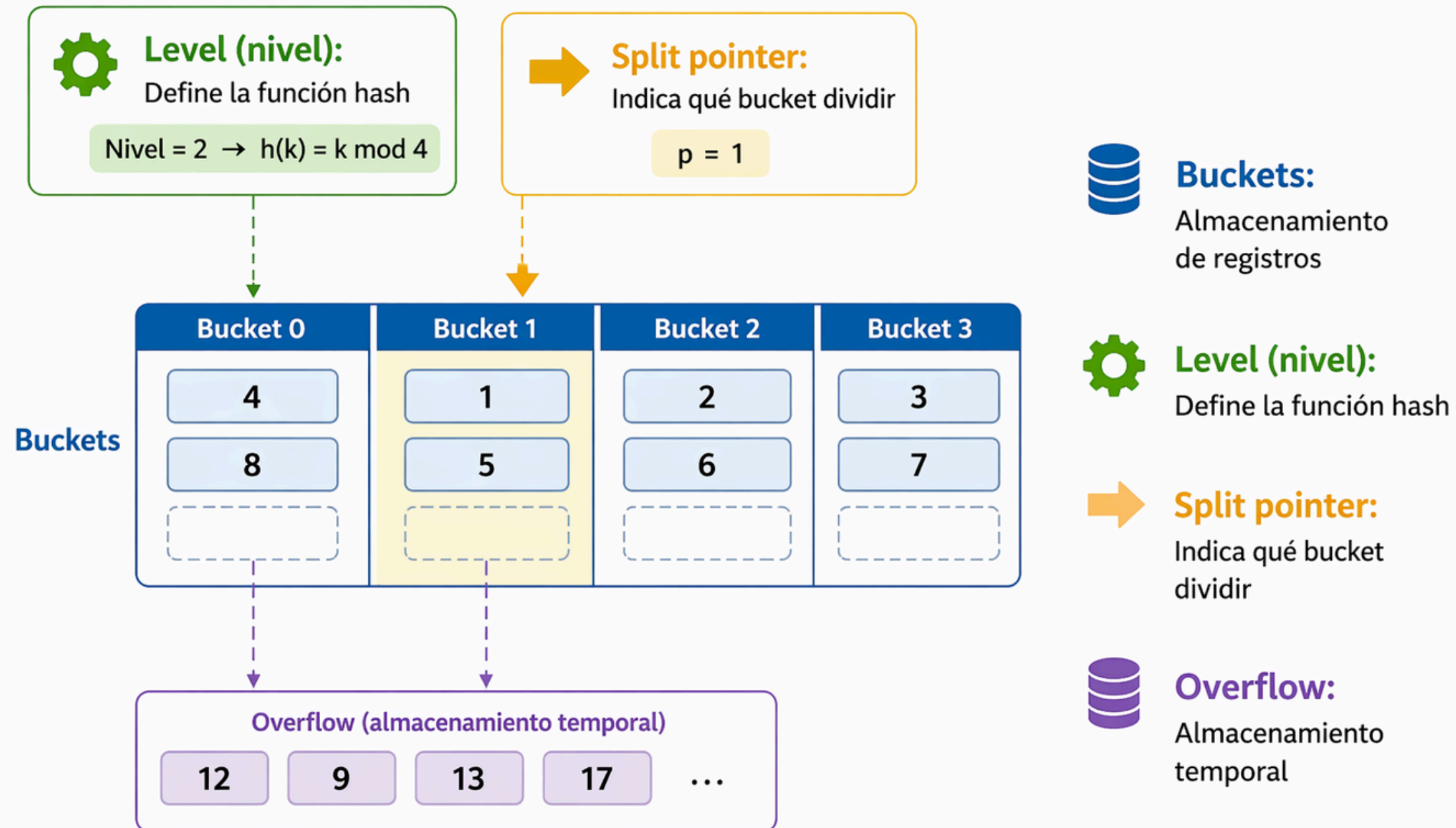


HASHING LINEAL

El hashing lineal, propuesto por Witold Litwin en 1980, adopta un enfoque fundamentalmente diferente al eliminar completamente el directorio. En su lugar, crece de forma progresiva y utiliza un mecanismo de división controlada mediante un puntero cíclico.

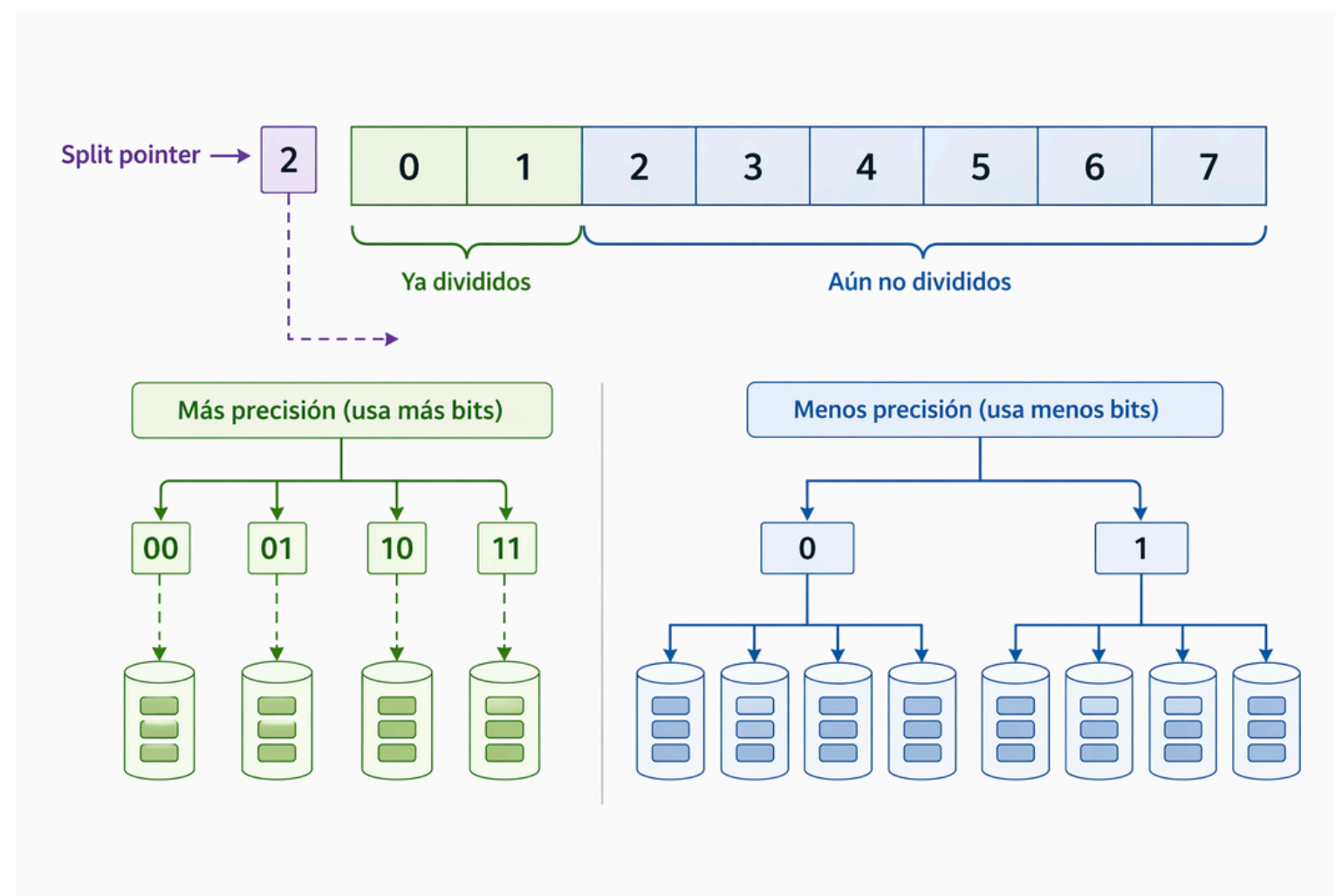


ESTRUCTURA DE DATOS



FUNCIÓN DE DIRECCIONAMIENTO

$b = h(k) \bmod (2^{\text{level}})$
if $b < \text{splitptr}$:
 $b = h(k) \bmod (2^{(\text{level}+1)})$



EJEMPLO

Level = 2 $\rightarrow 2^2 \Rightarrow$ 4 **buckets**

Split pointer = 2

Dato 1: $k = 6$

$6 \bmod 4 = 2$

$2 < 2 \rightarrow$ **No**

Bucket 2

$b = h(k) \bmod (2^{\text{level}})$

if $b < \text{splitptr}$:

$b = h(k) \bmod (2^{(\text{level}+1)})$

Dato 2: $k = 5$

$5 \bmod 4 = 1$

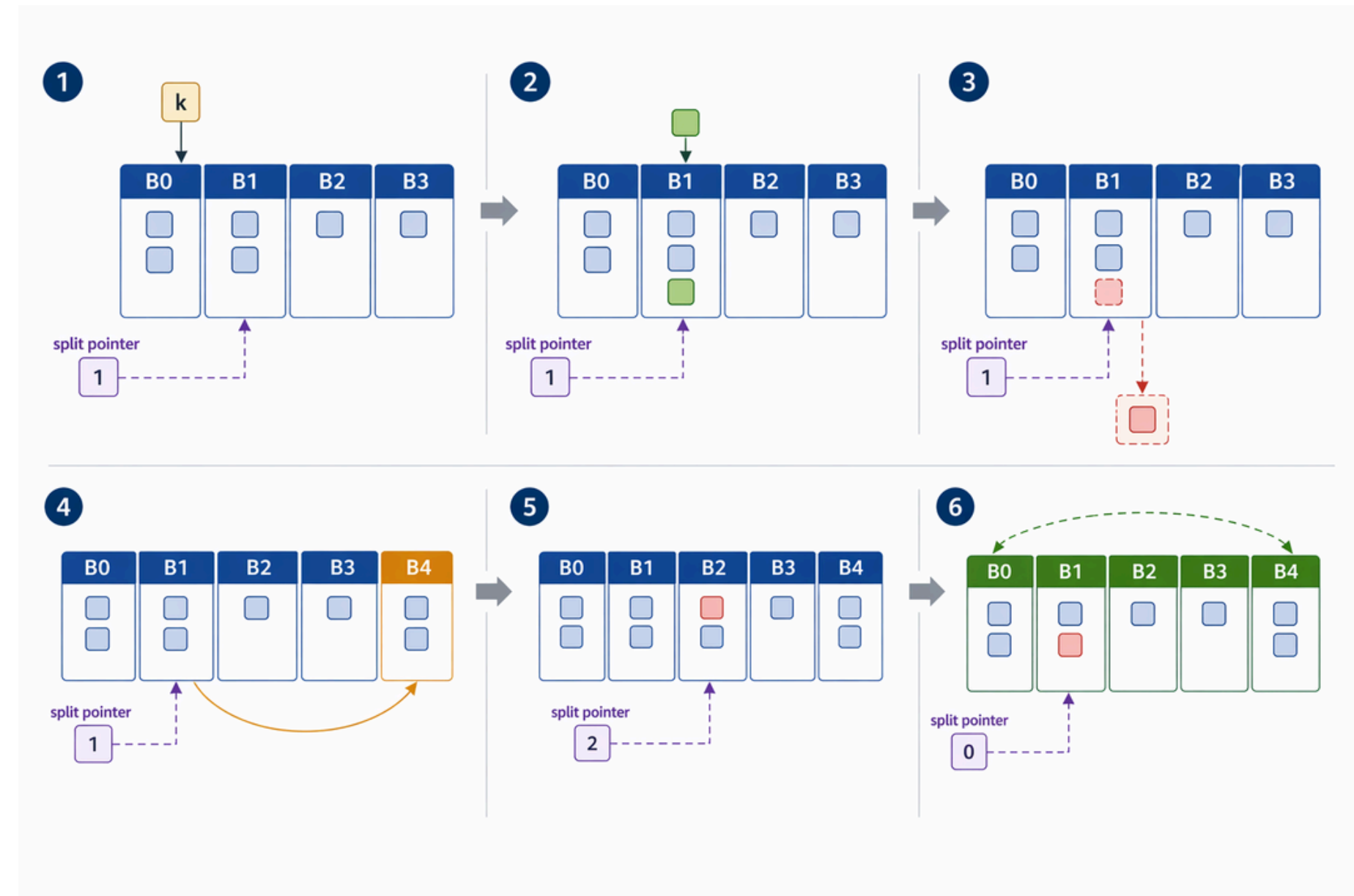
$1 < 2 \rightarrow$ **Sí**

$5 \bmod 8 = 5$

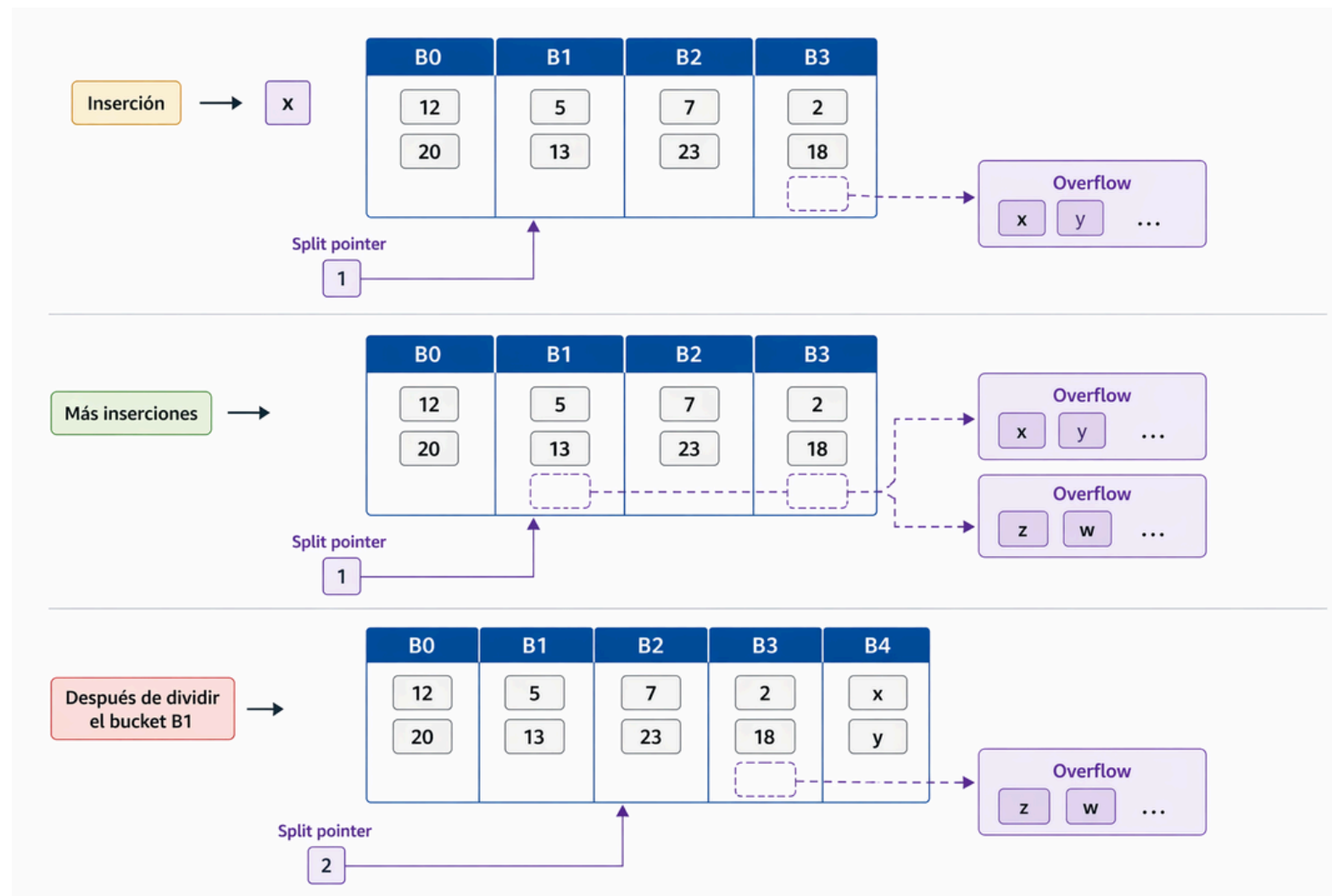
Bucket 5

MECANISMO DE CRECIMIENTO

1. Se calcula el bucket destino
2. Se inserta el registro
3. Puede ocurrir overflow
4. Se divide el bucket indicado por split pointer
5. El puntero avanza
6. Si completa el ciclo → aumenta el nivel



MANEJO DE DESBORDAMIENTOS



- Uso de buckets de overflow
- Manejo temporal de datos excedentes
- Se reorganizan al dividir el bucket

COMPORTAMIENTO DEL SISTEMA

La división ocurre cuando el sistema necesita crecer (factor de carga / overflow)

Se divide el bucket indicado por el split pointer

El bucket se divide aunque no esté lleno

Los buckets se numeran desde 0 (0, 1, 2, 3, ...)

El level cambia al completar un ciclo de divisiones

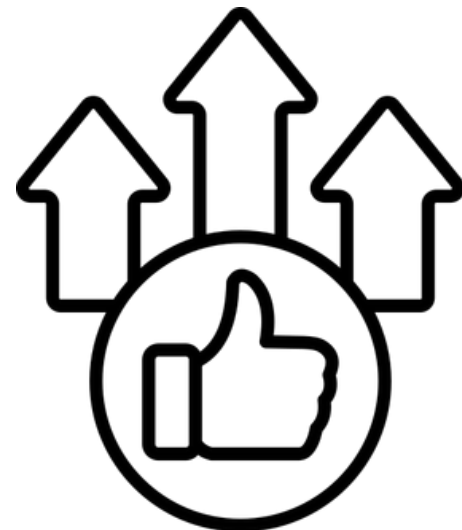
VENTAJAS

No requiere directorio

Crecimiento gradual

Buen uso de memoria

Comportamiento predecible

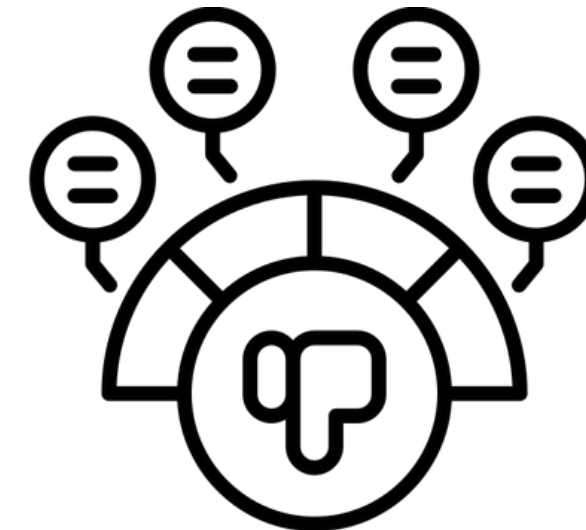


DESVENTAJAS

Uso de overflow

Más accesos en algunos casos

Mayor complejidad



ANÁLISIS COMPARATIVO: EXTENSIBLE VS. HASHING LINEAL

HASHING

CARACTERÍSTICA	HASHING EXTENSIBLE	HASHING LINEAL
ESTRUCTURA AUXILIAR	DIRECTORIO (Array) [Requiere Memoria RAM]	SIN DIRECTORIO [Solo Buckets en Disco]
MODO DE CRECIMIENTO	EXPONENCIAL (Se duplica el directorio entero)	LINEAL (Se añade 1 bucket a la vez)
DISPARADOR DE DIVISIÓN	SE DIVIDE EL BUCKET QUE SE LLENÓ (Inmediato)	SE DIVIDE EL BUCKET SEÑALADO POR EL "PUNTERO CÍCLICO" (Independiente de cuál se llenó)
MANEJO DE COLISIONES	REORGANIZACIÓN INMEDIATA (Se redistribuye usando más bits)	USO DE BUCKETS DE DESBORDAMIENTO (Cadenas temporales en disco)
USO DE MEMORIA RAM	ALTO (Directorio puede crecer a 2^i)	BAJO (No requiere tabla de índices)
Nº ACCESOS A DISCO (I/O típica)	1-2 (Muy estable)	1-2 + OVERFLOW (Degrada si hay cadenas largas)

ENTENDIENDO LAS DIFERENCIAS CLAVE

- **Encadenamiento:**

- Simple y natural.
- Degradación significativa con cadenas largas o factor de carga alto.
- Puede sufrir con distribución sesgada de claves.

- **Hashing Extensible:**

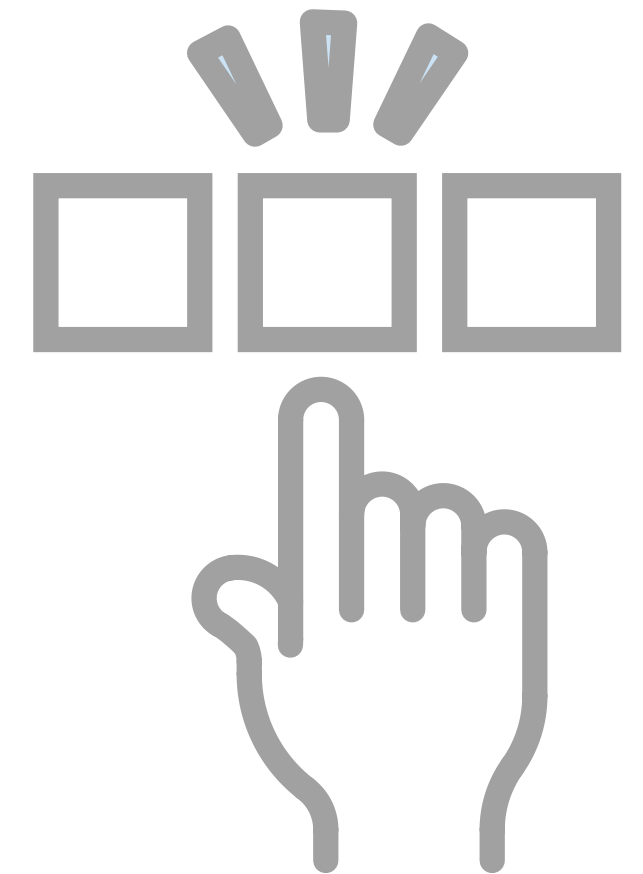
- Solución ordenada al crecimiento.
- Separa espacio de direcciones hash del espacio de buckets reales.
- **Ventaja:** Cambios localizados (solo en la cubeta dividida). Búsquedas con pocos accesos.
- **Costo:** Mantenimiento de un directorio que puede duplicarse.

- **Hashing Lineal:**

- Evita el directorio.
- Expansión incremental y gradual.
- **Desventaja:** Puede depender de registros de desbordamiento hasta que el puntero de división alcance los buckets afectados.
- **Nota:** Más difícil de entender inicialmente, pero atractivo para implementación dinámica.

¿QUÉ DICEN LOS ESTUDIOS Y CUANDO USAR CADA UNO?

- El hashing extensible ofrece aproximadamente 5% mejor utilización de almacenamiento.
- Ambos mantienen factores de carga entre 70-90% con rendimiento aceptable.
- **Extensible:** Ventaja en entornos con abundante memoria principal.
- **Lineal:** Preferible cuando la memoria es limitada o el directorio no cabe en RAM.



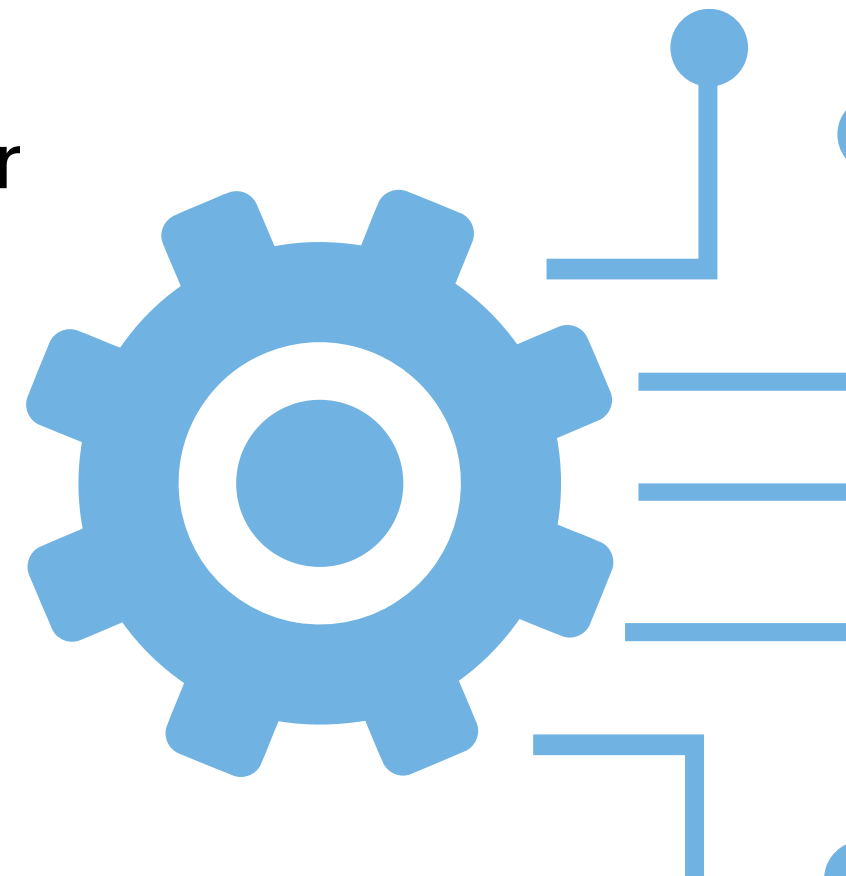
GUÍA RÁPIDA DE SELECCIÓN



Prefiera HASHING EXTENSIBLE cuando:	Prefiera HASHING LINEAL cuando:
La memoria principal es abundante.	La memoria principal es limitada o costosa.
Se requiere rendimiento absolutamente predecible (1-2 accesos).	La aplicación puede tolerar ocasionalmente accesos adicionales.
La aplicación es sensible a la latencia.	Se requiere crecimiento muy gradual sin pausas perceptibles.
El archivo es pequeño o mediano.	El archivo puede crecer hasta tamaños muy grandes.
	La aplicación es interactiva o de tiempo real.

PANORAMA DE LA INVESTIGACIÓN: OTROS ESQUEMAS RELEVANTES

- **Spiral Storage (Martin, 1979):** Función de mapeo espiral para expansión incremental.
- **Bounded Index Exponential Hashing (Lomet, 1983):** Limita el crecimiento del directorio usando una estructura de árbol.
- **Linear Virtual Hashing:** Combina hashing lineal con direccionamiento virtual.
- **Cuckoo Hashing:** Ofrece tiempo de acceso constante en el peor caso (principalmente en RAM, adaptado a disco).

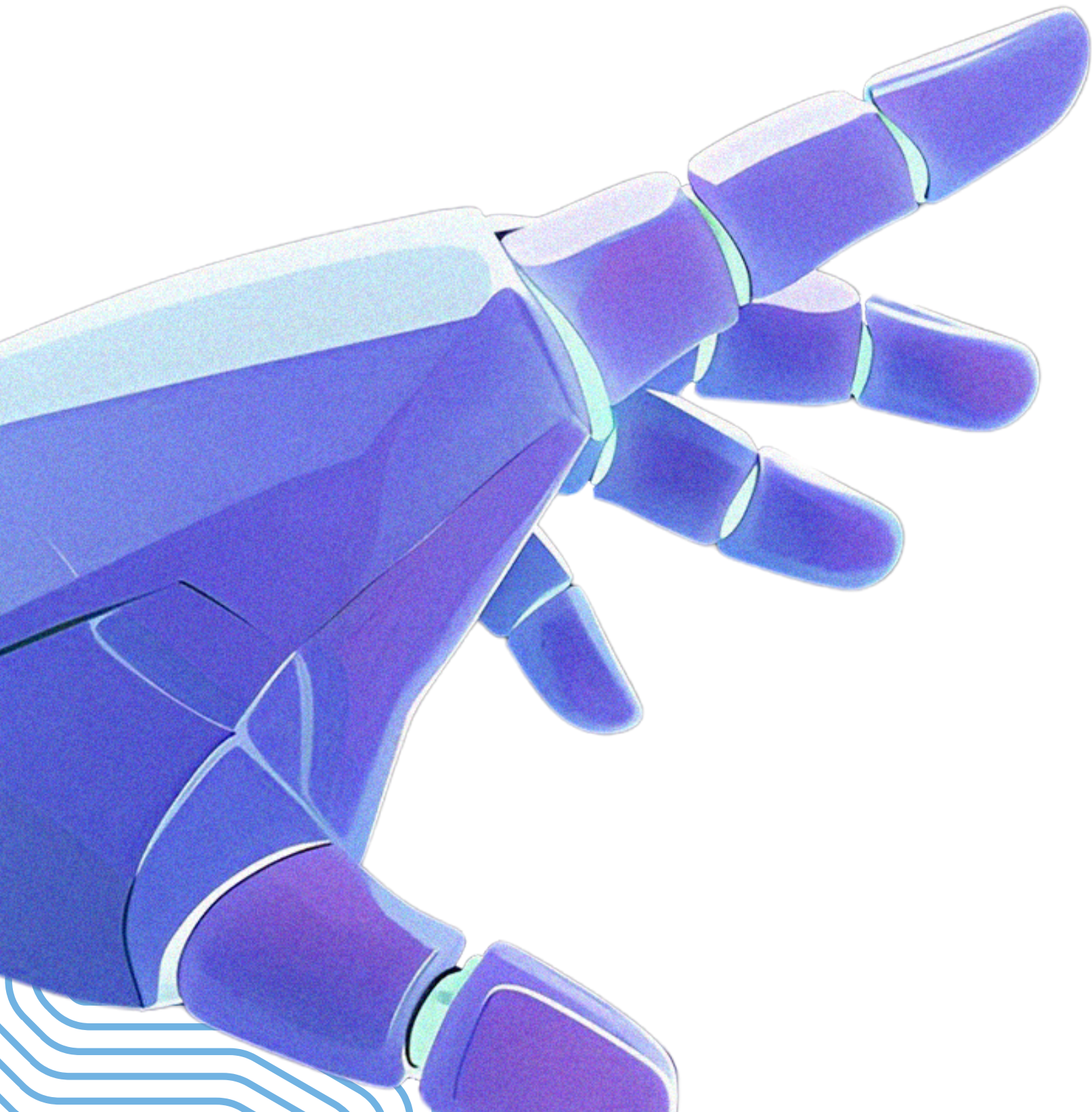


¿DÓNDE SE USA EL HASHING DINÁMICO?

- **Sistemas de Gestión de Bases de Datos (SGBD):**
 - Índices hash para búsquedas por igualdad exacta.
 - Organización de archivos heap.
 - Tablas hash en memoria para catálogos.
- **Sistemas de Archivos y Almacenamiento:**
 - Asignación de bloques.
 - Tablas de localización de objetos en almacenamiento distribuido.
 - Gestión de caché de buffers.
- **Aplicaciones en Memoria Principal:**
 - Tablas hash en lenguajes de programación (Ej. hashtables en Haskell).
 - Aplicaciones interactivas que no toleran pausas por rehashing.
 - Sistemas embebidos con restricciones de memoria.



EL FUTURO DEL HASHING DINÁMICO



- **Almacenamiento No Volátil (NVM):**
 - Estructuras híbridas que aprovechan acceso rápido similar a RAM con persistencia inmediata.
- **Sistemas Distribuidos:**
 - Hashing Consistente para Tablas Hash Distribuidas (DHT) y balanceo de carga en clusters NoSQL.
- **Aprendizaje Automático e Índices Aprendidos:**
 - Modelos de ML para predecir distribuciones de claves y optimizar funciones hash.
 - Reemplazo o aumento de índices tradicionales con modelos aprendidos.

CONCLUSIONES

- **Superación de limitaciones estáticas:**
 - Eliminan reorganizaciones masivas mediante crecimiento incremental controlado.
- **El encadenamiento NO es una solución completa para disco:**
 - No mantiene rendimiento constante en almacenamiento secundario.
- **Extensible y Lineal son opciones complementarias:**
 - **Extensible:** Rendimiento más predecible (costo: RAM para directorio).
 - **Lineal:** Minimiza requisitos de memoria (costo: manejo de desbordamientos).
- **Factores de carga del 70–90% son alcanzables manteniendo 1–2 accesos a disco.**
 - Rendimiento muy superior al encadenamiento puro en disco.
- **La elección del esquema depende del contexto:**
 - **Memoria abundante** → Hashing Extensible
 - **Memoria limitada / Archivos masivos** → Hashing Lineal
 - **Tablas en RAM** → Encadenamiento optimizado
 - **Sistemas interactivos** → Hashing Lineal (evita pausas)
- **Evolución continua:**
 - La investigación sigue activa, adaptándose a nuevas arquitecturas de hardware y sistemas distribuidos a escala global.

REFERENCIAS

- Bilkent University Computer Engineering Department. "Dynamic Hashing." CS351 Lecture Notes, 2010.
- Koushik, Murlidhar; Diehr, George. "Linear-density hashing with dynamic overflow sharing." Information Systems, Vol. 17, No. 5, pp. 359–380, 1992. Zbl 0781.68051.
- "Dynamic hashing strategies." In: Springer Book Chapter 6. ABES Repository.
- Koushik, M.; Diehr, G. "Linear-density hashing with dynamic overflow sharing." ScienceDirect, Information Systems, Vol. 17, Issue 5, pp. 359–380, September 1992.
- "Dynamic hashing schemes." ACM Computing Surveys, Vol. 20, Issue 2, June 1988. DOI: 10.1145/46157.
- Haskell Documentation. "Data.HashTable.ST.Linear." hasktables package version 1.0.1.3.
- "Dynamic hashing schemes." Scilit Publication Database.
- Haskell Documentation. "Data.HashTable.ST.Linear." hasktables package version 1.2.4.2.
- Priti, Tailor; Rustom, Morena. "A Survey of Hashing Techniques and Its Applicability for Efficient Buffer Cache Management." Journal of Global Research in Computer Sciences, Vol. 8, pp. 01–05, 2017.